

PixelMind

Offline AI-Powered Personal Photo Intelligence System

PC / Laptop Edition — 100% Local — Zero Internet Required

Semantic Search	Face Grouping	Relationship Graph	Privacy First
-----------------	---------------	--------------------	---------------

1. Executive Summary

PixelMind is a fully local, AI-powered photo management system for PC and laptop. It understands your photos semantically, groups faces automatically, builds a social relationship graph, and lets you search using plain English — without sending a single byte to the cloud.

Core Promise: Every AI model, every database, every computation runs on your own machine. No API keys. No subscriptions. No internet dependency — ever.

2. Problem Statement

Modern users accumulate tens of thousands of photos, but existing tools fail them in two key ways:

2.1 Discovery Problem

- Finding a specific photo requires remembering the exact date or album name.
- Manual tagging is time-consuming and inconsistent.
- Keyword search fails for visually-defined content like "beach sunset" or "birthday cake".

2.2 Privacy Problem

- Cloud photo apps (Google Photos, iCloud, Amazon Photos) upload your images to remote servers.
- Face recognition is performed on third-party infrastructure.
- Usage data, metadata, and social patterns are harvested by tech corporations.

Gap: No offline-first, AI-powered photo tool delivers Google Photos-level intelligence while keeping all data strictly local.

3. Solution Overview

PixelMind uses pre-downloaded AI models to index and understand photos at ingestion time. All intelligence is stored in local databases and is queryable through a desktop UI at any time — online or offline.

Capability	How It Works	Runs On
Semantic Image Search	CLIP converts photos and queries to vectors; cosine similarity matches	Local CPU/GPU
Face Detection	MTCNN detects all faces in every photo	Local CPU
Face Recognition	FaceNet converts face crops to 128-D embeddings	Local CPU/GPU
Face Clustering	DBSCAN groups embeddings into person-clusters	Local CPU
Relationship Graph	NetworkX builds a person-to-person co-appearance graph	Local RAM
Metadata Indexing	EXIF parser extracts date, GPS, camera model	Local CPU
Duplicate Detection	Perceptual hash (pHash) flags near-identical images	Local CPU

4. System Architecture

The system is divided into two runtime flows: the Indexing Pipeline (runs once per photo) and the Query Engine (runs on every search or browse action).

4.1 Indexing Pipeline (Storage Flow)

Triggered whenever new photos are added. Each stage is idempotent — partially indexed photos can be re-processed safely.

Stage	Input	Process	Output	Tech
1. Ingest	Folder path	Scan for .jpg .png .heic .webp	File list + hash	pathlib + hashlib
2. Preprocess	Raw image	Decode, resize 224x224, normalize	Normalized tensor	Pillow + NumPy
3. CLIP Embed	224x224 tensor	CLIP ViT-B/32 image encoder	512-D vector	CLIP (local weights)
4. Face Detect	Full-res image	MTCNN multi-scale detection	Bounding boxes	facenet-pytorch
5. Face Embed	Face crop 160x160	FaceNet Inception-ResNet	128-D embedding	facenet-pytorch
6. EXIF Extract	Raw image file	Parse EXIF/XMP metadata	Date, GPS, Camera	Pillow ExifTags
7. pHash	8x8 grayscale	DCT-based perceptual hash	64-bit hash	imagehash
8. Store	All outputs	Write to FAISS, SQLite, Neo4j	Persist DB state	FAISS+SQLite+NX

4.2 Query Engine — Three Modes

Mode A — Semantic Text Search

- User types e.g. "festival with fireworks"
- CLIP text encoder converts query to 512-D vector
- FAISS approximate nearest-neighbour search ranks results by cosine similarity

Mode B — Metadata Filter Search

- Filters: date range, GPS bounding box, camera model
- SQLite query with indexed columns, joined with FAISS IDs

Mode C — People / Relationship Search

- "Photos with Rahul and Amit together"
- Graph engine intersects both person-node edge sets
- Only images containing both people are returned

Merge Strategy: All three modes produce image ID sets. AND/OR merge logic is applied, results are re-ranked by relevance, and a deduplicated list is returned to the frontend.

5. Data Layer — Storage Design

5.1 FAISS Vector Index

Parameter	Value	Reason
Index type	IVFFlat (nlist=256)	Balance of speed vs accuracy for 100K-1M vectors
Metric	Inner Product (cosine after L2-norm)	CLIP embeddings are unit-normalised
Dimension	512	CLIP ViT-B/32 output dimension
Storage format	faiss.index (binary file)	Memory-mapped, fast cold load
Face index	Separate FlatL2 index	Exact search; face sets are small (<50K)

5.2 SQLite Metadata Database

Table	Key Columns	Purpose
images	id, file_path, phash, width, height, indexed_at	Master photo registry
metadata	image_id, shot_date, lat, lon, camera_make, camera_model	EXIF/Geo data
faces	id, image_id, bbox_x, bbox_y, bbox_w, bbox_h, clip_face_id	Face detections
clusters	id, label, thumbnail_face_id, face_count	Named person clusters
embeddings_map	image_id, faiss_index_id	Bridge between SQLite and FAISS IDs
duplicates	image_id_a, image_id_b, phash_distance	Near-duplicate pairs

5.3 Relationship Graph (NetworkX)

Element	Type	Attributes
Person node	Node (cluster_id)	label, face_count, first_seen, last_seen
Image node	Node (image_id)	shot_date, location, thumbnail_path
APPEARS_IN	Edge: Person -> Image	confidence, face_bbox
CO_APPEARS	Edge: Person <-> Person	weight (co-appearance count), image_ids
SHOT_AT	Edge: Image -> Location	GPS coordinates, resolved place name

6. AI Models — Local Deployment

All models are downloaded once during installation and stored locally. They are loaded into RAM at startup and kept resident for zero-latency inference.

Model	Task	Size	Device	Latency
CLIP ViT-B/32	Image & text embedding	~338 MB	CPU / CUDA	~80ms/img

Model	Task	Size	Device	Latency
MTCNN	Face detection	~2 MB	CPU	~30ms/img
FaceNet (InceptionResNetV1)	Face embedding	~90 MB	CPU / CUDA	~15ms/face
DBSCAN (scikit-learn)	Face clustering	No weights	CPU	<1s/10K faces
imagehash pHash	Duplicate detection	No weights	CPU	<1ms/img

GPU Acceleration: If an NVIDIA GPU with CUDA is detected, CLIP and FaceNet automatically use it. Indexing speed improves 8-15x. The app runs perfectly on CPU-only machines — just slower during bulk indexing.

7. Feature Specifications

7.1 Semantic Image Search

- Open-vocabulary queries — any phrase CLIP was trained on works.
- No training or fine-tuning required; works out of the box.
- Example queries:
 - "Sunset over mountains"
 - "Birthday cake with candles"
 - "Group selfie outdoors"
 - "Documents or whiteboards"
- Image-to-image search also supported: 'Find more like this photo'.

7.2 Face Grouping & People Management

Clustering Process

- All face embeddings collected into matrix after initial indexing.
- DBSCAN runs with eps=0.6, min_samples=2 (tunable in Settings).
- Each cluster = one person. Outliers form their own cluster.

User Labelling Flow

- People tab shows grid of face cluster thumbnails.
- Click a cluster, type a name (e.g. "Mom", "Rahul", "Colleague Priya").
- Name saved to clusters table in SQLite immediately.
- User can merge two clusters or split one cluster.

7.3 Smart Filtering

Filter Type	UI Control	Backend Query
Date Range	Calendar picker (from / to)	WHERE shot_date BETWEEN ? AND ?
Year / Month	Dropdown selector	WHERE strftime('%Y-%m', shot_date) = ?

Filter Type	UI Control	Backend Query
Location	Map bounding-box OR city text	WHERE lat BETWEEN ? AND lon BETWEEN ?
Camera	Dropdown from detected cameras	WHERE camera_make = ? AND camera_model = ?
Person	Multi-select person chips	Graph query for image_ids
Duplicates	Toggle switch	JOIN duplicates table, show pairs

7.4 Duplicate Detection

- pHash computed for every image at index time.
- Hamming distance ≤ 8 flags two images as near-duplicates.
- Side-by-side comparison view with metadata differences.
- Bulk delete, keep-best button supported.

7.5 Relationship Graph Explorer

- Interactive D3.js force-directed graph in the desktop UI.
- Node size = number of photos the person appears in.
- Edge thickness = co-appearance count between two people.
- Click any node to see all photos featuring that person.
- Timeline slider filters the graph to a specific date range.

7.6 Incremental Indexing

- watchdog library monitors configured folders for changes.
- New files detected → added to indexing queue immediately.
- Deleted files → embeddings and metadata purged from all stores.
- Renamed/moved files → path updated in SQLite; embeddings preserved.

8. Full Technology Stack

8.1 Backend (Python Core)

Library	Version	Purpose
Python	3.10+	Core runtime
PyTorch	2.1+	Tensor ops, CLIP & FaceNet inference
openai/clip	Latest	CLIP model weights + tokenizer
facenet-pytorch	2.5+	MTCNN + FaceNet implementation
faiss-cpu / faiss-gpu	1.7+	Vector similarity search index
scikit-learn	1.3+	DBSCAN clustering algorithm

Library	Version	Purpose
Pillow	10+	Image decoding, resizing, EXIF parsing
imagehash	4.3+	Perceptual hash for duplicate detection
numpy	1.24+	Numerical array operations
SQLite3	built-in	Metadata and graph cache persistence
watchdog	3.0+	File system change monitoring
FastAPI	0.110+	Internal REST API between backend and UI
uvicorn	0.27+	ASGI server for FastAPI
networkx	3.2+	Social relationship graph engine
reverse_geocoder	1.5+	Offline GPS-to-city-name resolution

8.2 Frontend (Electron + React)

Technology	Purpose
Electron 30+	Cross-platform desktop shell (Windows, macOS, Linux)
React 18+	Component-based UI framework
Vite	Dev server and production bundler
TailwindCSS	Utility-first styling
D3.js	Interactive relationship graph visualisation
React Query	Async state management for FastAPI calls
Electron Builder	Packages into .exe (Win) / .dmg (Mac) / .ApplImage (Linux)

9. Detailed Implementation Plan

Phase 1 — Foundation (Weeks 1–2)

Task	Details	Deliverable
Project scaffold	Monorepo: /backend /frontend /models	Runnable empty project
Model downloader	Download CLIP, FaceNet, MTCNN to /models at first run	models/download.py
Image ingestion	pathlib scanner, MIME filter, SHA-256 dedup	ingest.py
Preprocessing	Decode, resize 224x224, normalise, handle HEIC/RAW	preprocess.py
SQLite schema	All tables + indexes on shot_date, lat, lon, cluster_id	schema.sql + db.py
FAISS index init	IVFFlat index, save/load to disk, insert/query helpers	vector_store.py

Phase 2 — AI Pipelines (Weeks 3–4)

Task	Details	Deliverable
CLIP pipeline	Batch process images (batch=32), write to FAISS + embeddings table	clip_pipeline.py
CLIP text search	Tokenize query, encode, cosine search FAISS, return top-K	search.py
MTCNN detection	Run MTCNN on full-res, store bboxes in faces table	face_detect.py
FaceNet embedding	Crop + align faces, batch-encode, write to face FAISS index	face_embed.py
DBSCAN clustering	Load all face embeddings, run DBSCAN, write cluster_id back	face_cluster.py
EXIF extraction	Parse EXIF, GPS rational-to-decimal, write to metadata table	exif.py
pHash dedup	Compute pHash for all images, Hamming check, write duplicates	dedup.py

Phase 3 — Graph + API (Week 5)

Task	Details	Deliverable
Graph builder	Build NetworkX DiGraph from clusters+faces+images, CO_Appearance	graph_builder.py
Graph queries	co_appearance_count(), shared_photos(), people_in_photo()	graph_queries.py
FastAPI endpoints	GET /search, /people, /graph, /duplicates; POST /label, /index	api/routes.py
File watcher	watchdog Observer, debounce 2s, push to indexing queue	watcher.py
Indexing queue	Threading queue, priority levels, progress via SSE to frontend	indexer.py

Phase 4 — Desktop UI (Weeks 6–7)

Task	Details	Deliverable
Electron shell	Main process spawns Python uvicorn, manages lifecycle + IPC	main.js
Search page	Search bar, results grid, lazy thumbnails, similarity badges	SearchPage.jsx
People page	Face cluster grid, click-to-label, merge/split controls	PeoplePage.jsx
Photo detail	Full-res view, EXIF panel, face overlays, similar photos	PhotoDetail.jsx
Graph explorer	D3.js force-directed graph, nodes, edges, timeline slider	GraphExplorer.jsx
Duplicates page	Side-by-side pairs, bulk delete, keep-best button	DuplicatesPage.jsx
Settings page	Folder management, model paths, DBSCAN tuning, GPU toggle	SettingsPage.jsx

Phase 5 — Packaging & Polish (Week 8)

Task	Details	Deliverable
Installer builder	electron-builder: .exe (NSIS), .dmg, .AppImage with Python bundle	electron-builder.yml
Bundled Python	PyInstaller spec for all Python deps + model weights	pyinstaller.spec
First-run wizard	Guide: select folder, download models, run initial index	Onboarding.jsx

Task	Details	Deliverable
Performance profiling	Profile indexing speed, tune batch sizes, identify bottlenecks	Profile report
Error handling	Graceful failures for corrupt images, missing EXIF, low-disk	All modules
User testing	Test on Windows 10/11 + Ubuntu 22.04 with 10K photo libraries	QA report

10. Performance Targets & Hardware Requirements

10.1 Minimum Hardware

Component	Minimum	Recommended
CPU	Intel i5 8th gen / Ryzen 5 3600	Intel i7 12th gen / Ryzen 7 5800
RAM	8 GB	16 GB
Storage	10 GB free (models + DB)	SSD with 50 GB free
GPU	Not required	NVIDIA GTX 1660+ with CUDA 11.8
OS	Windows 10 64-bit / Ubuntu 20.04 / macOS 11.0	Windows 11 / Ubuntu 22.04 / macOS 14+

10.2 Indexing Speed Benchmarks

Hardware	Images / Second	Time for 10,000 Photos
Intel Core i5 (CPU only)	~1.2 images/sec	~2.3 hours
Intel Core i7 (CPU only)	~2.5 images/sec	~1.1 hours
NVIDIA GTX 1660 + i7	~15 images/sec	~11 minutes
NVIDIA RTX 3070 + i9	~35 images/sec	~5 minutes

Note: Indexing runs only once per photo. Search queries always return in under 200ms regardless of library size, because FAISS is pre-built and memory-resident.

11. Privacy & Security Design

Principle	Implementation
Zero outbound network calls	No HTTP calls to external servers. FastAPI binds to 127.0.0.1 only.
No telemetry or analytics	No crash reporting, usage tracking, or analytics libraries included.
All AI models local	CLIP and FaceNet weights stored in /models. No remote inference API used.
Encrypted database option	SQLCipher-encrypted SQLite available as opt-in for sensitive libraries.
No cloud sync	No Dropbox, OneDrive, or any cloud integration. Data stays on local disk.
Open source	Full codebase on GitHub. Users can audit every line.

Principle	Implementation
Offline reverse geocoding	GPS-to-city via local reverse_geocoder library with bundled GeoNames data.

12. Risks & Mitigations

Risk	Severity	Mitigation
Slow indexing on old hardware	High	Background processing; app usable while indexing; configurable batch size
DBSCAN splits same person into 2 clusters	Medium	Manual merge UI; re-run DBSCAN with tuned eps parameter
EXIF missing in phone photos	Medium	Fallback to file modification date; ML-based scene date estimation planned
FAISS + SQLite ID sync drift on delete	High	Atomic deletion wrapper removes from both stores in single transaction
Large libraries exceed RAM	Medium	FAISS memory-mapped mode; NetworkX paged to disk beyond 1M edges
Electron bundle size too large	Medium	PyInstaller + Electron ~600MB; NSIS installer with optional GPU extras

13. Future Roadmap

Version	Feature
v1.1	Natural language photo editing: 'Remove red-eye from all birthday photos'
v1.2	Local LLM (LLaMA 3 8B via llama.cpp) for conversational photo Q&A
v1.3	Scene classification using local EfficientNet (Beach, Indoor, Night, Sports)
v1.4	Auto-album generation — group by event using temporal + scene clustering
v2.0	Local generative search: 'Show me a photo similar to this sketch'
v2.1	NAS / external drive support with portable index that travels with the drive

14. Project Directory Structure

Path	Contents
pixelmind/	Project root
pixelmind/backend/	All Python source code
backend/pipelines/	clip_pipeline.py, face_detect.py, face_embed.py, face_cluster.py, exif.py, dedup.py
backend/api/	FastAPI app, routes, Pydantic schemas
backend/db/	schema.sql, db.py (SQLite helper), vector_store.py (FAISS helper)
backend/graph/	graph_builder.py, graph_queries.py
backend/utils/	preprocess.py, ingest.py, watcher.py, indexer.py
pixelmind/frontend/	Electron + React source
frontend/src/pages/	SearchPage, PeoplePage, PhotoDetail, GraphExplorer, Duplicates, Settings

Path	Contents
frontend/src/components/	Shared UI components (ImageGrid, FaceCard, FilterBar, etc.)
frontend/electron/	main.js (Electron main process), preload.js
pixelmind/models/	Downloaded model weights — not in git, fetched at first run
pixelmind/data/	Runtime data — pixelmind.db, faiss indexes, thumbnails — gitignored
pixelmind/scripts/	download_models.py, migrate_db.py, benchmark.py
pixelmind/dist/	Built installer output — .exe, .dmg, .ApplImage

Appendix — Glossary

Term	Definition
CLIP	Contrastive Language-Image Pretraining — OpenAI model that maps images and text to the same vector space.
FaceNet	Google model that maps a face crop to a 128-D vector; similar faces have small Euclidean distance.
MTCNN	Multi-task Cascaded Convolutional Network — fast, accurate face detection pipeline.
FAISS	Facebook AI Similarity Search — library for efficient nearest-neighbour search over dense vectors.
DBSCAN	Density-Based Spatial Clustering — clusters points without knowing the number of clusters in advance.
pHash	Perceptual Hash — a hash of visual content; similar images have similar pHash values.
Embedding	A dense numerical vector encoding the semantic meaning of an image, face, or text phrase.
Cosine Similarity	Dot product of two unit-normalised vectors. 1.0 = identical meaning, 0.0 = unrelated.
IVFFlat	Inverted File Flat — FAISS index type that partitions vectors into clusters for fast approximate search.

— End of Document —